

Can a CNN Learn Snake in a Hallucinated Environment?

A Small-Scale Study of Visual World-Model Exploitation and Grounding

Anonymous

Abstract

Interactive video world models suggest a route to training agents without querying the true environment at every step. This paper studies a deliberately small version of that problem: train an action-conditioned visual world model for Snake, train CNN policies only inside that learned model, and evaluate the resulting policies in the true code simulator. The experiment is designed to expose reward hacking rather than hide it. The world model predicts the next RGB frame, scalar reward, terminal probability, and an unclamped floating-point snake length from a short visual context and an action. Terminal states are not encoded by tinting the RGB frame; death and win signals are separate scalar targets. We report hallucinated return, real-simulator return, and their gap across world-model scale, context length, and CNN policy size. We also evaluate an iterative grounding loop in which a policy trained inside the world model generates action traces, those traces are replayed in the true simulator to collect correct visual transitions, and the existing world model is fine-tuned rather than retrained from scratch.

1 Introduction

Generative interactive environments such as Genie [1] and real-time streaming game world models such as Matrix-Game 2.0 [2] make it plausible to use learned video dynamics as training environments. For reinforcement learning, however, a learned environment is also an attack surface: if the policy can discover model inaccuracies that produce reward inside the model but do not transfer to the real environment, apparent progress can be negative transfer.

We study this question in Snake because the true simulator is cheap, deterministic, and unambiguous. That lets us measure the gap between imagined performance and real performance without arguing about evaluator noise. The core question is not whether Snake can be solved by a CNN. It can. The question is narrower: when a CNN policy is trained only against a learned visual dynamics model, does it learn behavior that transfers back to the real simulator, or does it exploit the model?

2 Method

Environment. The simulator renders a $128 \times 128 \times 3$ RGB Snake board. There is no HUD, score text, keyboard overlay, or terminal-state color tint. This matters because an earlier pilot introduced a global terminal tint, which made the visual prediction task confounded with death detection. In v2, terminal information is available only through the world model’s scalar terminal head.

World model. The world model is action-conditioned and autoregressive. Given the previous $k \in \{1, 2, 5\}$ frames, previous reward, and the next discrete action, it predicts:

$$\hat{x}_{t+1}, \quad \hat{r}_{t+1}, \quad \hat{d}_{t+1}, \quad \hat{\ell}_{t+1} \tag{1}$$

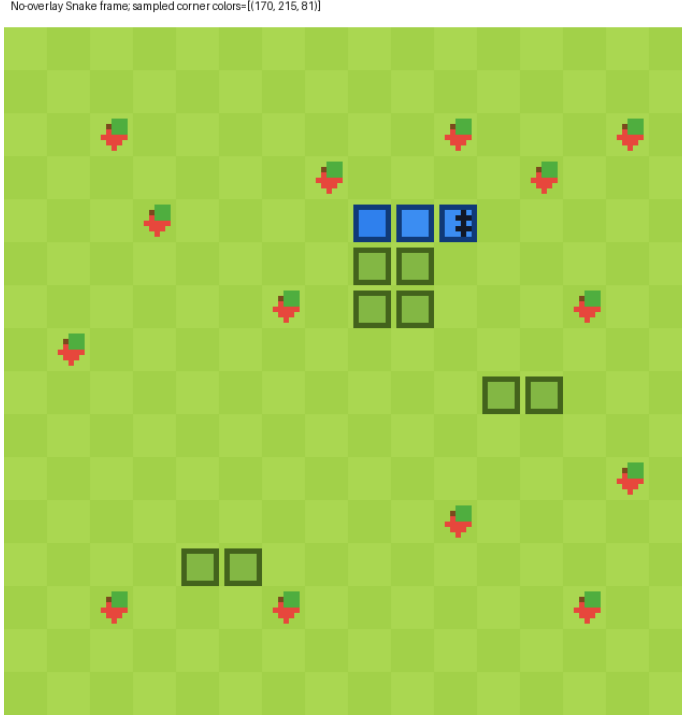


Figure 1: Example v2 frame. The board is rendered as RGB pixels only; terminal state is not painted into the frame.

where $\hat{x}_{t+1}$ is the next RGB frame, $\hat{r}_{t+1}$ is scalar reward, $\hat{d}_{t+1}$ is a terminal logit, and $\hat{\ell}_{t+1}$ is a floating-point snake length prediction. The length output is intentionally not clamped, so large or implausible deviations remain measurable.

Policy training. For each frozen world model, CNN policies of three sizes are trained with PPO [3] inside hallucinated rollouts. Episodes in the learned environment terminate when the predicted terminal probability crosses a threshold or when the rollout reaches a maximum length. Policies are then evaluated in both the learned world model and the true simulator.

Grounding loop. The grounding experiment is designed to approximate settings where real data are expensive but not impossible to obtain. After training a policy inside the world model, we execute that policy’s actions in the real simulator, collect a small set of correct transitions, merge those transitions with the original random/safe/greedy dataset, and fine-tune the existing world model checkpoint. We repeat this loop several times and measure whether the hallucinated-real return gap shrinks. This is closer to deployment-time grounding than training a new model from scratch on policy-generated data.

3 Experimental setup

The main dataset contains 50,000 real simulator transitions from a mixture of random, safe-random, straight, and greedy policies. The main sweep trains two world-model sizes, three context lengths, and three CNN policy sizes, yielding 18 world-model/policy settings. The W&B project is linked at [snake-hallucinated-worlds-v2](#).

WM	Ctx	CNN	Hall.	Real	Gap	Apples
wm_2m	5	small	7.401	6.000	1.401	6.000
wm_1m	1	small	0.395	-1.000	1.395	0.000
wm_1m	1	large	0.356	-1.000	1.356	0.000
wm_2m	5	medium	5.469	6.000	-0.531	6.000
wm_2m	1	small	-1.904	-1.000	-0.904	0.000
wm_1m	1	medium	-1.980	-1.000	-0.980	0.000
wm_2m	1	large	-2.410	-1.000	-1.410	0.000
wm_1m	2	large	7.529	9.000	-1.471	10.000
wm_1m	2	small	4.950	7.000	-2.050	8.000
wm_2m	1	medium	-3.105	-1.000	-2.105	0.000
wm_1m	5	small	8.927	12.000	-3.073	12.000
wm_1m	5	medium	7.877	11.000	-3.123	12.000
wm_1m	2	medium	6.908	12.000	-5.092	12.000
wm_2m	2	large	10.120	18.000	-7.880	13.000
wm_2m	5	large	8.831	18.000	-9.169	13.000
wm_2m	2	medium	8.481	18.000	-9.519	13.000
wm_1m	5	large	7.749	18.000	-10.251	13.000
wm_2m	2	small	7.521	18.000	-10.479	13.000

Table 1: Main transfer results. Gap is hallucinated return minus real-simulator return; a positive gap indicates the policy performed better inside the world model than in the true simulator.

Iter.	New data	Total data	Hall.	Real	Gap
1	1000	1000	-0.024	-1.000	0.976
2	440	1440	-8.300	0.000	-8.300
3	1000	2440	1.073	0.000	1.073

Table 2: Iterative grounding results. Each iteration collects a small amount of real simulator data using the previous policy’s actions and fine-tunes the existing world model.

4 Results

The tables and plots in this section are generated from synced experiment artifacts using `python -m snake_wm_v2.export_paper_assets`.

5 Discussion

This study is intentionally small, but the exploitation mechanism is the point. If a CNN trained inside a learned Snake model obtains positive hallucinated return and negative real return, then the learned environment is not just inaccurate; it is exploitable by the policy optimizer. The most important quantity is therefore not frame MAE alone, but the transfer gap between imagined and real returns.

The grounding loop tests a practical mitigation. If the policy finds a model exploit, replaying its actions in the true simulator reveals the correct transitions near that exploit. Fine-tuning the world model on a small number of these transitions should, in principle, make the exploit harder

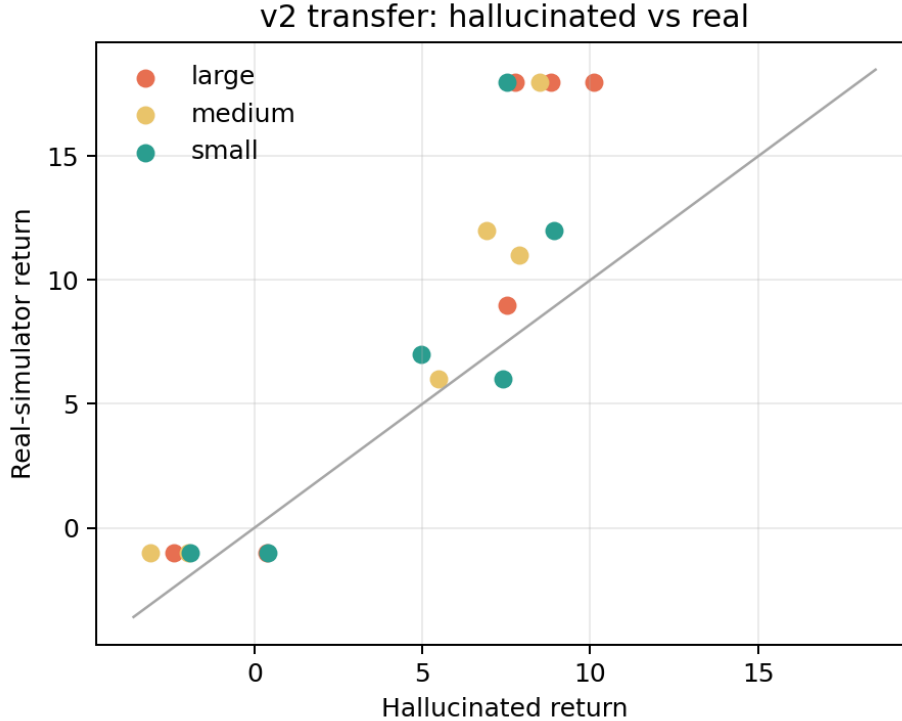


Figure 2: Hallucinated return versus real-simulator return. Points below the diagonal indicate overestimation by the learned world model.

to repeat. The experiment does not prove that this scales to humanoids or open-world games. It provides a controlled measurement harness for the same mechanism: train in imagination, evaluate in reality, then ground the model where the policy drove it out of distribution.

6 Limitations

Snake is discrete, low-dimensional, and rendered from complete state. The results should not be interpreted as evidence that small CNN policies or small latent world models are sufficient for richer domains. The dataset is also policy-biased; random/safe/greedy trajectories do not cover every state that a trained optimizer may induce. Finally, the current experiments use a single simulator and a modest compute budget, so conclusions should be framed as a reproducible case study rather than a general scaling law.

7 Reproducibility

The code, configs, generated figures, and selected result artifacts are packaged in the accompanying repository. The main config is `configs/focused_v2.json`. The local smoke test is `scripts/local_smoke.sh`. Experiment metrics are logged to the linked W&B project.

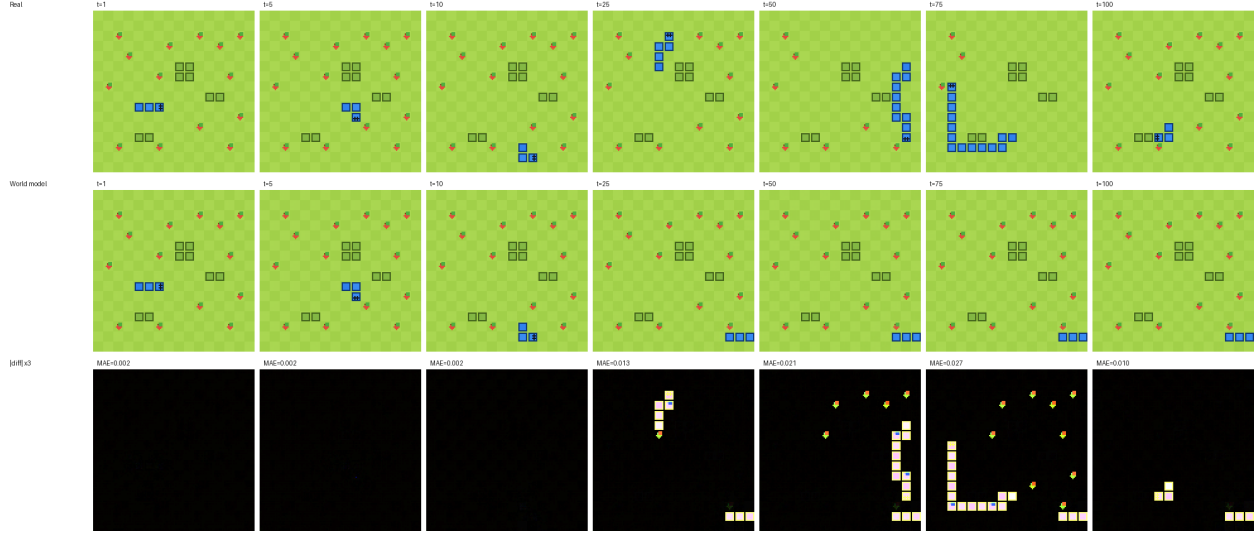


Figure 3: Autoregressive world-model rollout against simulator frames. The difference row visualizes pixel-level drift.

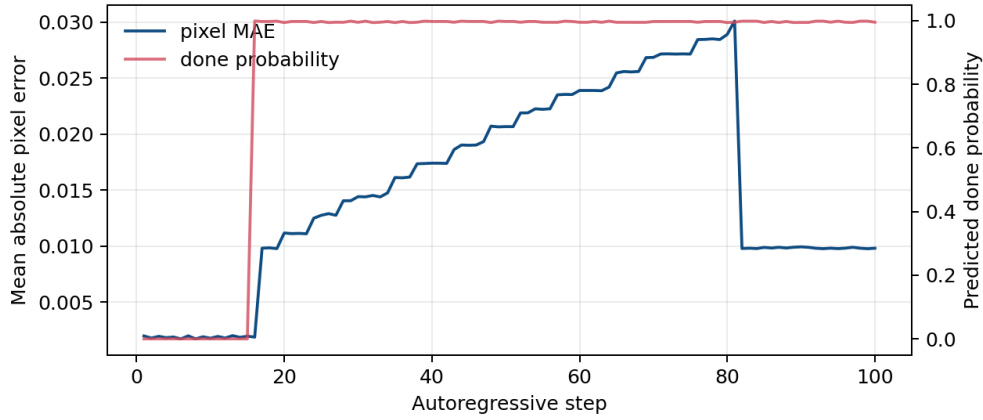


Figure 4: Pixel error and predicted terminal probability across an autoregressive rollout.

References

- [1] Jake Bruce, Michael D. Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, Yusuf Aytar, Sarah Bechtle, Feryal Behbahani, Stephanie C. Y. Chan, Nicolas Heess, Lucy Gonzalez, Simon Osindero, Sherjil Ozair, Scott Reed, Jingwei Zhang, Konrad Zolna, Jeff Clune, Nando de Freitas, Satinder Singh, and Tim Rocktaschel. Genie: Generative interactive environments. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- [2] Xianglong He, Chunli Peng, Zexiang Liu, Boyang Wang, Yifan Zhang, Qi Cui, Fei Kang, Biao Jiang, Mengyin An, Yangyang Ren, Baixin Xu, Hao-Xiang Guo, Kaixiong Gong, Cyrus Wu, Wei Li, Xuchen Song, Yang Liu, Eric Li, and Yahui Zhou. Matrix-game 2.0: An open-source, real-time, and streaming interactive world model. *arXiv preprint arXiv:2508.13009*, 2025.

- [3] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.